

11 Build y Versioning Guide

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

11 Build y Versioning Guide	3
title: "Cartridge & Cloud — Build y Versioning Guide" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" application_version_reference: "0.0.26" persistence_schema_reference: "IntegratedGameStateSnapshot schema 2" documentary_baseline_reference: "v0.6"	4
Cartridge & Cloud — Build y Versioning Guide	4
0. Propósito	4
0.1. Preguntas que debe responder cada build	4
0.2. Alcance	4
0.3. Fuera de alcance inmediato	5
1. Jerarquía de autoridad	5
1.1. Verdad implementada frente a objetivo	6
1.2. Fecha de la fotografía	6
2. Fotografía técnica vigente	6
2.1. Registro de build de cierre de Sprint 16	6
2.2. Discrepancias activas	7
2.2. Ausencia de automatización Unity	7
3. Los ejes de versión	7
3.1. Prohibición de colapsar ejes	8
3.2. Identificador completo de una build	8
4. SemVer adaptado al proyecto	8
4.1. Periodo 0.x	8
4.2. Incremento PATCH	9
4.3. Incremento MINOR	9
4.4. Incremento MAJOR	9
4.5. Prerelease y metadata	9
5. Estado y ciclo de una versión	10
5.1. Congelación	10
5.2. Una versión, varias builds	10
6. Cambios incompatibles	10
6.1. Clases de breaking change	10
6.2. Procedimiento	11
6.3. Prohibición de invalidación silenciosa	11
7. Schema de persistencia y compatibilidad	11
7.1. Regla de incremento	11
7.2. Matriz de compatibilidad	11
7.3. Lectura y escritura	12
7.4. Build candidate y saves	12
8. Versionado de contenido	12
8.1. Objetivo	12
8.2. Cambios peligrosos	12
8.3. Regla	12
9. Versionado documental	13
9.1. Baselines	13
9.2. Cadencia	13
9.3. Relación con una build	13
10. Repositorio, ramas y working copy	13
10.1. Rama estable	13
10.2. Ramas	13
10.3. Working copy limpia	14
10.4. SHA	14
10.5. Tags	14
11. Tipos de build	14

11.1. Development	14
11.2. QA	15
11.3. Candidato de hito	15
11.4. Release	15
12. Build Profiles	15
12.1. Perfil observado Windows_Development	15
12.2. Perfiles objetivo	16
12.3. Ownership	16
12.4. Cambio de perfil	16
13. Escenas y orden de arranque	16
13.1. Regla permanente	16
13.2. Estado observado	16
13.3. Objetivo Sprint 16	17
13.4. Distribución	17
13.5. Validación	17
14. Plataforma Windows x64	17
14.1. Baseline	17
14.2. Archivos mínimos esperados	17
14.3. Resolución y ventana	17
14.4. Ruta de guardado	18
15. Scripting backend y runtime	18
15.1. Mono en desarrollo	18
15.2. IL2CPP	18
15.3. Prohibición	18
15.4. Stripping	18
16. Flags de desarrollo, debugging y profiling	19
16.1. Builds de profiling	19
16.2. Logging	19
17. Nomenclatura de builds	19
17.1. Formato	19
17.2. Reglas	19
17.3. Build ID interno	20
18. Estructura de salida	20
18.1. Separación de staging	20
19. Metadata y manifiesto de build	20
19.1. Campos mínimos	21
53. Plantilla de release record	21
54. Historial de builds verificado	22
54.1. Lecciones	22
55. Deuda y evolución recomendada	22
Prioridad inmediata	22
Antes de H6	23
Antes de demo/Alpha	23
Antes de Release	23
56. Taxonomía de fallos de build	23
56.1. Fallo de compilación C#	23
56.2. Fallo de importación o asset	24
56.3. Fallo de Scene List	24
56.4. Fallo de Player distinto del Editor	24
56.5. Fallo de empaquetado o hash	25
56.6. Fallo de almacenamiento	25
57. Entorno y hardware del build	25
57.1. Registro del entorno	25
57.2. Hora y fecha	26
57.3. Máquina de build	26
58. Procedimiento de bump de versión	26
58.1. Bump ordinario de sprint	26
58.2. Bump por hotfix	26
58.3. Bump de schema	27

58.4. Bump documental	27
58.5. Bump de contenido	27
59. Matriz de artefactos por gate	27
59.1. Regla de proporcionalidad	28
60. Escenarios de migración y actualización	28
60.1. Nueva instalación	28
60.2. Actualización compatible	28
60.3. Actualización con migración	28
60.4. Save futuro	28
60.5. Cambio de Application Identifier	28
60.6. Demo a juego completo	29
61. Revisión formal de un candidato	29
61.1. Reunión o acto de gate	29
61.2. Preguntas de rechazo	29
61.3. Cierre	29
62. Runbook de recuperación	30
62.1. La build candidata no arranca	30
62.2. El hash publicado no coincide	30
62.3. Se incluyó un secreto	30
62.4. Corrupción de saves tras update	30
62.5. Build profile dañado	30
62.6. Paquete o Unity cambió accidentalmente	31
Anexo A. Rutas relevantes	31
Anexo B. Valores protegidos	31
Anexo C. Definiciones	31
Anexo D. Ruta final	32
Actualización de baseline y build W0	32
Estado operativo vigente tras W0	32
Baseline de remediación	32
Regla de cierre y no propagación	32

11 Build y Versioning Guide

Metadatos normativos v2.0
document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

Nota de coherencia v1.1: las reglas de tiempo, pausa, apertura, cierre, liquidación y autosave diario se interpretan conforme a ADR-0086 y ADR-0087. Los estados de sprint o baseline anteriores se mantienen únicamente como fotografía histórica cuando una actualización posterior los sustituye.

title: "Cartridge & Cloud — Build y Versioning Guide" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" application_version_reference: "0.0.26" persistence_schema_reference: "IntegratedGameStateSnapshot schema 2" documentary_baseline_reference: "v0.6"

Cartridge & Cloud — Build y Versioning Guide

0. Propósito

Este documento define la política autoritativa para identificar, producir, validar, empaquetar, archivar y distribuir builds de **Cartridge & Cloud**. También define cómo se relacionan las versiones de aplicación, documentación, persistencia, contenido, motor, paquetes, artefactos y repositorio.

La guía debe impedir cinco errores frecuentes:

1. llamar "release" a una carpeta local no identificada;
2. cambiar la versión visible sin registrar el código que la produjo;
3. tratar un cambio de schema como si fuera un simple cambio cosmético;
4. aprobar una build porque Unity mostró Build Succeeded sin ejecutarla fuera del Editor;
5. publicar un artefacto que no pueda asociarse de forma inequívoca a un commit, configuración, conjunto de escenas, pruebas y known issues.

La guía no convierte automáticamente el proyecto en una producción con CI, firma digital o SteamPipe. Describe la norma actual y la evolución preparada. Cuando una capacidad todavía no existe, se marca como **objetivo**, **propuesta** o **deuda**, nunca como implementación cerrada.

0.1. Preguntas que debe responder cada build

Toda build relevante debe permitir responder:

- ¿qué código exacto la produjo?;
- ¿qué versión de Unity y paquetes se usó?;
- ¿qué versión de aplicación muestra?;
- ¿qué schema de guardado entiende?;
- ¿qué escenas contiene y en qué orden?;
- ¿qué perfil y opciones de compilación se aplicaron?;
- ¿qué pruebas se ejecutaron antes y después?;
- ¿qué recorrido externo se validó?;
- ¿qué defectos permanecen abiertos?;
- ¿dónde están el ejecutable, el log y la evidencia?;
- ¿puede reproducirse o, como mínimo, auditarse?;
- ¿puede retirarse o reemplazarse sin confundir a testers o usuarios?;

0.2. Alcance

La norma cubre:

- builds locales de desarrollo;
- builds QA;
- candidatos de Sprint 16, Sprint 17 y H6;

- demo, Alpha, Beta, Release Candidate y Release futuras;
- versionado SemVer adaptado;
- schema y migraciones;
- perfiles de Unity;
- Windows x64;
- nomenclatura y estructura de salida;
- pruebas y aceptación;
- `Player.log`, crash logs y diagnóstico;
- checksums, archivo y retención;
- tags, releases y changelog;
- rollback y hotfix;
- automatización futura;
- preparación para Steam sin asumir que Steamworks esté implementado.

0.3. Fuera de alcance inmediato

No se considera actualmente implementado:

- pipeline CI de Unity;
- firma de código para Windows;
- instalador MSI/MSIX;
- Steamworks SDK;
- SteamPipe;
- depots o branches de Steam;
- actualización diferencial;
- crash reporting remoto;
- telemetría de builds;
- símbolos públicos de depuración;
- distribución para macOS, Linux o consolas;
- un perfil Release final aprobado;
- IL2CPP como backend autorizado para distribución.

Estas capacidades pueden incorporarse mediante roadmap, spike, ADR y QA específicos.

1. Jerarquía de autoridad

La guía aplica el siguiente orden:

1. 00_Enfoque_y_Alcance.md;
2. 01_Game_Design_Document.md;
3. 02_Vertical_Slice_Specification.md;
4. 03_Technical_Design_Document.md;
5. 04_Modelo_de_Datos.md;
6. 05_UX_Flow.md;
7. 06_Production_Roadmap_y_Sprint_Plan.md;
8. 07_QA_Testing_Plan.md;
9. 08_QA_Testing_Matrix.xlsx;
10. 09_CSharp_Coding_Standards.md;
11. 10_Unity_Project_Setup_Guide.md;
12. esta guía consolidada;
13. ProjectSettings, Build Profiles, paquetes y scripts reales;
14. ADR y registros operativos;
15. Build & Versioning Guide v0.6;
16. versiones v0.5, v0.4 y v0.3.

Cuando una política histórica es menos estricta que un gate actual, prevalece el gate actual. Por ejemplo, ADR-0010 permite omitir la revisión rutinaria de `Player.log` en un sprint ordinario sin

incidencias; sin embargo, Sprint 16, Sprint 17 y H6 exigen expresamente build externa y revisión del log. Para esos gates se aplica la exigencia superior.

1.1. Verdad implementada frente a objetivo

Se usan estas etiquetas:

Etiqueta	Significado
OBSERVADO	valor leído en los archivos suministrados
VALIDADO	valor respaldado por registro de ejecución
VIGENTE	norma aprobada en este documento
OBJETIVO	configuración que debe implementarse antes de un gate
DIFERIDO	previsto para una fase posterior
REQUIERE ADR	cambio estructural que no puede aplicarse informalmente

1.2. Fecha de la fotografía

La fotografía técnica original corresponde al **1 de julio de 2026** y fue actualizada con el cierre operativo del **6 de julio de 2026**. Un SHA, versión o estado no debe reutilizarse como si describiera automáticamente una working copy posterior.

2. Fotografía técnica vigente

Campo	Estado observado
Proyecto	Cartridge & Cloud
Compañía	VRM Games
Unity	6000.3.18f1 (5eb53e4c07)
URP	17.3.0
Plataforma objetivo	Windows x64
Versión de aplicación	0.0.26
Application Identifier	com.vrmgames.cartridgeandcloud
Input	Input System (activeInputHandler: 1)
Player log	activado
Compilación determinista C#	activada
Backend de desarrollo	Mono por baseline/ausencia de override
Compresión Development	LZ4
Build Profile	Windows_Development
Escenas validadas para cierre	Bootstrap, MainMenu, StoreInitial
Escena objetivo del slice	StoreInitial
Resolución por defecto observada	1024 × 768
Ventana redimensionable observada	desactivada
Schema de sesión mínimo	1
Schema integrado	2
HISTORICAL EVIDENCE — infraestructura automatizada retirada:	Tests documentados
Estado	Sprints 0-15 cerrados; Sprint 16 COMPLET

2.1. Registro de build de cierre de Sprint 16

- Build ID lógico: BLD-016-POST.

- Versión: 0.0.26.
- Plataforma: Windows x64 Development.
- Arranque: Bootstrap → MainMenu → StoreInitial. HISTORICAL EVIDENCE — las pruebas automatizadas existentes en esa ejecución finalizaron en PASS; la infraestructura no está presente actualmente y se reimplantará en una fase futura.
- Golden Path externo: PASS.
- Persistencia tras cierre y reapertura: PASS.
- Player.log: revisado sin errores bloqueantes.
- Estado: build aceptada para cerrar Sprint 16; no es candidata H6 ni artefacto público.

2.2. Discrepancias activas

La configuración observada todavía no representa el objetivo final de Sprint 16:

- el Build Profile incluye Store, no StoreInitial;
- la lista global también incluye Store;
- TestLab está incluido en el perfil de desarrollo;
- no se encontró un script de build del Player en Assets o Tools;
- la build se produce actualmente mediante flujo manual de Unity;
- no existe pipeline CI aportado;
- no existe perfil QA/H6/Release separado;
- no existe metadata de build embebida confirmada;
- buildNumber.Standalone permanece en 0;
- la resolución 1024 × 768 no debe interpretarse como objetivo comercial aprobado;
- no hay evidencia de firma, SteamPipe o distribución pública.

Ninguna discrepancia debe “corregirse” fuera de su gate. La migración Store → StoreInitial se realiza después de conectar y validar la nueva escena, no antes.

2.2. Ausencia de automatización Unity

La inspección del paquete suministrado no encontró una clase Editor dedicada a producir el Player ni un script de línea de comandos para Unity. Tools/Blender/.../build_all.py pertenece al kit de Blender y no es un pipeline del ejecutable. Por tanto, los ejemplos de automatización incluidos en esta guía son **diseño de referencia**, no descripción de una herramienta ya disponible.

3. Los ejes de versión

Cartridge & Cloud no tiene una única “versión”. Tiene varios ejes que deben registrarse por separado.

Eje	Ejemplo	Autoridad	Qué identifica
Aplicación	0.0.26	PlayerSettings.bundleVersion	evolución jugable/técnica
Build	Build003	registro de artefacto	iteración concreta de una mis
Git	SHA de 40 caracteres	repositorio	código y assets exactos
Schema	2	snapshot/codec	forma persistente compatible
Contenido	content manifest futuro	catálogo	conjunto de definiciones/asse
Documento	1.0 / baseline v0.6	front matter/baseline	norma o fotografía documenta
Unity	6000.3.18f1	ProjectVersion.txt	toolchain
Paquetes	URP 17.3.0, etc.	manifest/lock	dependencias resueltas
Steam	BuildID futuro	Steamworks	subida aprobada por Valve

3.1. Prohibición de colapsar ejes

No se debe afirmar:

- “schema 0.0.26”;
- “build v0.6” sin aclarar si v0.6 es documentación;
- “release d54316” como sustituto de una versión de aplicación;
- “Steam build 0.3.0” sin registrar el BuildID real;
- “última build” sin Build ID, fecha, SHA y ubicación.

3.2. Identificador completo de una build

Una build queda identificada por la tupla:

ApplicationVersion
+ BuildIteration
+ CommitSHA
+ UnityVersion
+ BuildProfile
+ Platform/Architecture
+ SchemaVersion
+ BuildDateUTC

Para un candidato de hito se añaden:

ArtifactSHA256
+ TestReport
+ PlayerLogReview
+ KnownIssuesSnapshot
+ GateDecision

4. SemVer adaptado al proyecto

Se usa MAJOR.MINOR.PATCH, sin prefijo v dentro de PlayerSettings. El prefijo v se reserva para tags o nombres humanos cuando resulte útil.

4.1. Periodo 0.x

Mientras MAJOR = 0, el proyecto sigue en desarrollo previo al contrato comercial estable. Esto no autoriza cambios arbitrarios: cada cambio incompatible de datos, experiencia o distribución debe documentarse.

La secuencia inicial observada usa el patch para representar el cierre de sprints:

Sprint	Versión	Capacidad cerrada
0	0.0.1	fundación y primera build
1	0.0.2	Bootstrap y flujo de escenas
2	0.0.3	sesión, slots y save skeleton
3	0.0.4	movimiento, cámara e input
4	0.0.5	grid y placement
5	0.0.6	shell de tienda y acceso
6	0.0.7	productos e inventario
7	0.0.8	pedidos y recepción
8	0.0.9	displays y reposición
9	0.0.10	perfiles y spawning de clientes
10	0.0.11	shopping y reservas
11	0.0.12	cola y checkout

Sprint	Versión	Capacidad cerrada
12	0.0.13	ciclo diario y cierre
13	0.0.14	economía y resultados
14	0.0.15	save/load integrado
15	0.0.16	integración UI/UX
16	0.0.26	presentación representativa y build externa cerradas en PASS
17	por congelar	estabilización y H6

La versión de Sprint 17 no se inventa en esta guía. Debe congelarse al abrir el sprint y comprobar que no colisiona con hotfixes o builds de Sprint 16.

4.2. Incremento PATCH

Se incrementa PATCH para:

- cierre ordinario de sprint dentro de la línea 0.0.x;
- hotfix que no añade un bloque de producto;
- corrección de build distribuida manteniendo compatibilidad;
- revisión interna que cambia código, assets o datos ejecutables.

Un cambio puramente documental no obliga a modificar `bundleVersion`, aunque sí modifica la versión o hash documental.

4.3. Incremento MINOR

Durante 0.x, MINOR se reserva para una transición de madurez o contrato de producto, por ejemplo:

- vertical slice aprobado;
- MVP de tienda aprobado;
- demo pública;
- Alpha funcional;
- Beta feature complete.

La decisión debe registrarse en el gate. No se aplica solo porque el número de patch resulte alto.

4.4. Incremento MAJOR

1.0.0 corresponde al primer release comercial estable aprobado. Un 2.0.0 futuro indicaría una ruptura de contrato pública de gran escala, no una migración interna cualquiera.

4.5. Prerelease y metadata

Para nombres de artefacto o canales puede usarse:

0.3.0-h6.1
0.4.0-alpha.2
0.4.0-beta.5
1.0.0-rc.1

Unity `bundleVersion` debe mantenerse simple y verificarse en el Player. La metadata adicional puede vivir en un manifiesto de build y en el nombre del artefacto.

5. Estado y ciclo de una versión

Estado	Significado
PROPOSED	versión candidata no congelada
FROZEN	número reservado para un sprint/gate
BUILT	existe al menos una build identificada
VALIDATING	artefacto bajo QA
ACCEPTED	gate aprobado
SUPERSEDED	reemplazada por otra build/versión
WITHDRAWN	retirada por defecto o error de publicación
RELEASED	distribuida al canal previsto

5.1. Congelación

Antes de producir una build de cierre:

1. confirmar objetivo y gate;
2. resolver la versión propuesta;
3. actualizar `PlayerSettings.bundleVersion`;
4. registrar schema compatible;
5. comprobar que el changelog contiene solo cambios verificables;
6. impedir cambios de alcance no relacionados;
7. registrar el SHA que se pretende validar.

5.2. Una versión, varias builds

Una versión puede tener varias iteraciones:

0.0.26 / Build001 – preintegración representativa
0.0.26 / Build002 – primera conexión de StoreInitial
0.0.26 / Build003 – corrección de clic sobre UI
0.0.26 / Build004 – candidato de cierre de Sprint 16

No se incrementa la versión por cada intento fallido. Sí se incrementa el contador de build cuando se conserva o comunica el artefacto.

6. Cambios incompatibles

Un cambio es incompatible si puede impedir que una build anterior o posterior interprete estado, contenido, comandos o distribución de forma segura.

6.1. Clases de breaking change

- cambio de schema persistente;
- cambio semántico de un campo existente;
- eliminación o reutilización de un ID estable;
- modificación del ownership de inventario o reservas;
- cambio de unidad monetaria;
- cambio del orden o identidad de escenas requeridas;
- cambio de Application Identifier o ruta de guardado;
- cambio de backend o stripping que elimina código requerido;
- cambio de catálogo que deja referencias persistentes huérfanas;
- cambio de contrato de input o cierre que hace una partida no operable;
- cambio de API pública de herramientas de build consumidas por CI.

6.2. Procedimiento

Todo breaking change requiere:

1. descripción y motivación;
2. impacto en saves, tests, builds y rollback;
3. nueva versión de schema cuando corresponda;
4. migrador o declaración explícita de incompatibilidad;
5. fixtures de la versión anterior;
6. prueba de upgrade;
7. prueba de rechazo de versiones futuras;
8. actualización de known issues y release notes;
9. ADR si afecta arquitectura o distribución;
10. plan de recuperación.

6.3. Prohibición de invalidación silenciosa

No se puede borrar, ignorar o reemplazar un save incompatible sin informar. En builds internas se puede declarar una ruptura controlada, pero debe aparecer en el registro antes de entregar el artefacto.

7. Schema de persistencia y compatibilidad

La baseline distingue:

- GameSessionSnapshot schema 1;
- IntegratedGameStateSnapshot schema 2.

El schema integrado es la ruta autoritativa objetivo del slice. La ausencia actual de placement dinámico en schema 2 se trata como deuda conocida, no como compatibilidad completa.

7.1. Regla de incremento

Se incrementa schema cuando cambia la estructura o semántica persistente. No se incrementa por:

- refactor interno sin cambio de serialización;
- corrección de UI;
- cambio visual no persistente;
- aumento de cobertura de tests;
- cambio documental.

7.2. Matriz de compatibilidad

Build	Schema mínimo legible	Schema escrito	Observación
pre-S14	variable/esqueleto	anterior	no usar como contrato actual
0.0.15+	según codec validado	2	snapshot integrado
futuro schema 3	1/2 si hay migrador	3	requiere fixtures y gate

La tabla debe actualizarse con evidencia real antes de una distribución externa.

7.3. Lectura y escritura

- una build no debe escribir un schema que no pueda validar;
- una build que detecte schema futuro debe rechazarlo sin mutación;
- la lectura debe completarse en memoria antes de aplicar estado;
- la escritura usa temporal, validación, promoción y backup;
- el primario corrupto no debe destruir un backup válido;
- un downgrade no se presupone compatible;
- probar “carga sin excepción” no basta: debe verificarse equivalencia observable.

7.4. Build candidate y saves

Antes de aprobar un candidato de hito:

- crear save nuevo;
 - guardar y cargar;
 - cerrar proceso y continuar;
 - probar backup;
 - probar corrupción controlada;
 - probar schema futuro no soportado;
 - comprobar tres slots;
 - verificar inventario, reservas, checkout, día y economía;
 - registrar versión y schema en evidencia.
-

8. Versionado de contenido

Los assets y catálogos pueden cambiar sin modificar la forma del snapshot, pero aun así romper una partida. Por ello se prepara un content manifest version independiente.

8.1. Objetivo

El manifiesto futuro debe identificar:

- productos;
- mobiliario;
- proveedores;
- perfiles de clientes;
- audio/VFX requeridos;
- escenas y catálogos;
- definiciones de balance;
- hashes o versión del conjunto.

8.2. Cambios peligrosos

- borrar un ID persistido;
- reasignar un ID a otro concepto;
- cambiar una huella de mueble persistido;
- retirar un producto con stock existente;
- renombrar una escena sin migrar referencias;
- modificar una definición de precio que deba permanecer congelada en una transacción.

8.3. Regla

Los IDs se deprecian antes de eliminarse. Un catálogo nuevo debe poder:

- resolver el ID antiguo;

- migrarlo;
 - sustituirlo por fallback explícito;
 - o rechazar la carga con diagnóstico claro.
-

9. Versionado documental

La versión documental no se sincroniza mecánicamente con la aplicación.

9.1. Baselines

Una baseline documental representa una fotografía aprobada. No debe reescribirse para ocultar historia. Las correcciones posteriores generan:

- nueva versión;
- changelog;
- hash;
- o registro operativo complementario.

9.2. Cadencia

- registros operativos: por sprint o ejecución;
- baseline oficial: por cierre de fase;
- baseline intermedia: solo ante cambio material de arquitectura, plataforma, tooling o dirección;
- esta guía: actualizar cuando cambia la política de build/versionado.

9.3. Relación con una build

Un candidato de hito debe enlazar la baseline documental que describe su comportamiento. Una baseline no prueba que el ejecutable corresponda a ella; la asociación exige SHA, registro de build y evidencia.

10. Repositorio, ramas y working copy

10.1. Rama estable

main es la referencia integrada. Una build candidata debe producirse desde:

- commit publicado y limpio;
- o commit candidato claramente identificado antes de la validación final.

No se aprueba un hito desde cambios locales no registrados.

10.2. Ramas

Convenciones permitidas:

```
feature/<tema>
fix/<tema>
docs/<tema>
release/<version>
hotfix/<version>-<tema>
```

En desarrollo individual pueden ser breves. No deben crear una falsa sensación de revisión: antes de integrar siguen siendo obligatorios compilación, tests y aceptación.

10.3. Working copy limpia

Preflight mínimo:

`git status`

Debe distinguirse:

- cambios intencionales incluidos;
- archivos generados ignorados;
- evidencia aún no publicada;
- cambios accidentales en escenas, settings o .meta;
- subidas incompletas.

10.4. SHA

Registrar SHA completa de 40 caracteres. Una abreviatura puede mostrarse en UI, pero el build record conserva la completa.

10.5. Tags

Los tags representan hitos, no cada sprint. Formato recomendado:

```
v0.0.1-project-foundation
v0.3.0-vertical-slice
v0.4.0-alpha.1
v1.0.0
```

Un tag:

- apunta al commit validado;
- no se mueve después de publicar;
- se crea después del PASS del gate;
- no se crea para un artefacto fallido;
- incluye release record cuando se distribuye.

11. Tipos de build

Tipo	Propósito	Audiencia	Artefacto retenido
Editor Test	desarrollo inmediato	desarrollador	no
Development	smoke y diagnóstico	interno	opcional
QA	regresión y campañas	QA interno/testers	sí durante campaña
H6 Candidate	aprobación del vertical slice	gate interno	sí
Private Tester	prueba controlada	testers seleccionados	sí
Demo	experiencia pública limitada	público	sí
Alpha	feature coverage	interno/externo limitado	sí
Beta	estabilización y contenido	testers	sí
RC	candidato a lanzamiento	release team	sí
Release	distribución pública	usuarios	sí y permanente
Hotfix	corrección urgente	canal afectado	sí

11.1. Development

- puede incluir TestLab;
- usa Mono y LZ4 mientras siga aprobada la baseline;

- permite Development Build;
- no requiere ZIP para cada iteración;
- debe ejecutarse externamente cuando cambia comportamiento Player;
- no se comunica como demo.

11.2. QA

- perfil separado objetivo;
- escenas de producción, sin herramientas innecesarias;
- logging suficiente;
- build identificada;
- resultados y defectos asociados;
- puede conservar símbolos privados según la investigación.

11.3. Candidato de hito

- checkout limpio;
- versión congelada;
- pruebas completas;
- Golden Path;
- Player.log revisado;
- checksum;
- paquete de evidencia;
- known issues congelados;
- decisión formal.

11.4. Release

No es simplemente un Development Build con el flag desmarcado. Requiere decisiones sobre:

- backend;
- stripping;
- símbolos;
- logging;
- crash reporting;
- firma;
- Steam;
- licencias;
- privacidad;
- soporte y rollback.

12. Build Profiles

12.1. Perfil observado Windows_Development

Campo	Valor
Asset	Assets/_Project/Settings/BuildProfiles/Windows_Development.asset
Target	Windows Standalone
Arquitectura	Intel 64-bit
Development	activado
Profiler autoconnect	desactivado
Deep Profiling	desactivado
Script Debugging	desactivado

Campo	Valor
Wait for Managed Debugger	desactivado
Copy PDB	desactivado
Create Solution	desactivado
Compression	LZ4
Override scene list	activado

12.2. Perfiles objetivo

Windows_Development
 Windows_QA
 Windows_H6_Candidate
 Windows_Demo
 Windows_Release

No es obligatorio crear todos ahora. Cada perfil se crea al abrir el gate que lo necesita.

12.3. Ownership

Los Build Profiles:

- se versionan;
- se revisan como configuración crítica;
- no se editan durante un build sin registrar el cambio;
- no deben depender de una lista global diferente sin explicación;
- deben tener un test o checklist de escenas;
- no contienen secretos.

12.4. Cambio de perfil

Cambiar backend, escenas, stripping, arquitectura, compression, development flags o player settings de distribución exige:

1. diff del asset;
2. motivo;
3. build antes/después;
4. regresión;
5. actualización de esta guía o ADR si es estructural.

13. Escenas y orden de arranque

13.1. Regla permanente

Bootstrap es la primera escena de toda build jugable.

13.2. Estado observado

0 – Bootstrap
 1 – MainMenu
 2 – Store
 3 – TestLab

El orden aparece tanto en la lista global como en Windows_Development.

13.3. Objetivo Sprint 16

Después de conectar StoreInitial y validar runtime:

```
0 – Bootstrap
1 – MainMenu
2 – StoreInitial
3 – TestLab      # solo Development
```

La escena Store se conserva temporalmente como fallback o referencia hasta cerrar la migración, pero no debe permanecer como destino ambiguo del flujo aprobado.

13.4. Distribución

Para QA de hito, demo, Alpha, Beta, RC y Release:

```
0 – Bootstrap
1 – MainMenu
2 – StoreInitial
```

TestLab se excluye salvo que exista una razón interna explícita. No debe ser accesible por el usuario final.

13.5. Validación

- cada ruta existe;
 - cada GUID es correcto;
 - no hay escenas deshabilitadas inesperadas;
 - MainMenu carga el destino previsto;
 - Continue restaura la escena correcta;
 - volver al menú no duplica ApplicationRoot;
 - Quit finaliza el proceso;
 - no se inicia directamente desde una escena de contenido en la build.
-

14. Plataforma Windows x64

14.1. Baseline

- target: Standalone Windows;
- arquitectura: x86_64;
- ejecutable: CartridgeAndCloud.exe;
- Application Identifier: com.vrmgames.cartridgeandcloud;
- ruta de datos Unity asociada a compañía/producto;
- Player log activado.

14.2. Archivos mínimos esperados

```
CartridgeAndCloud.exe
CartridgeAndCloud_Data/
UnityPlayer.dll
UnityCrashHandler64.exe
```

La lista exacta puede variar con backend, Unity y plugins. La validación no debe depender de una lista histórica rígida si el toolchain cambia de forma aprobada.

14.3. Resolución y ventana

El proyecto observa 1024 × 768 y ventana no redimensionable. Antes de H6 deben verificarse al menos 1280 × 720, 1920 × 1080 y 2560 × 1440 conforme a QA. Cambiar los valores por defecto requiere validar:

- HUD;
- modales;
- escala UI;
- cámara;
- placement;
- localización ES/EN;
- modo pantalla completa/ventana;
- persistencia de opciones si existe.

14.4. Ruta de guardado

La ruta depende de `companyName` y `productName`. Cambiar cualquiera puede aparentar pérdida de saves. Requiere ADR, migración y prueba en una instalación existente.

15. Scripting backend y runtime

15.1. Mono en desarrollo

Mono está aprobado para iteración de desarrollo. Ventajas dentro de la baseline:

- ciclo de build rápido;
- diagnóstico sencillo;
- coherencia con registros históricos;
- menor coste durante integración.

15.2. IL2CPP

IL2CPP está diferido. Antes de adoptarlo:

- crear ADR;
- medir tiempo de build;
- validar stripping;
- probar serialización y reflection;
- probar plugins;
- ejecutar suite completa;
- comparar rendimiento;
- revisar stack traces y símbolos;
- validar Steam Overlay cuando exista;
- conservar una ruta de rollback.

15.3. Prohibición

No cambiar a IL2CPP “porque es Release” sin realizar el gate. Un backend distinto puede descubrir fallos que no aparecen en Mono.

15.4. Stripping

El stripping actual no constituye una política Release cerrada. Cualquier aumento requiere pruebas sobre:

- serialización;
- tipos creados dinámicamente;
- JSON;
- reflection;
- ScriptableObjects;
- localización;

- addressables si se incorporan;
- código de plugins.

16. Flags de desarrollo, debugging y profiling

Opción	Development	QA	Candidate	Release
Development Build	sí	según campaña	no por defecto	no
Script Debugging	no por defecto	solo investigación	no	no
Autoconnect Profiler	no	solo sesión dedicada	no	no
Deep Profiling	no	solo build especial	no	no
Player Log	sí	sí	sí	política por decidir
PDB/símbolos	no baseline	privados si se aprueba	privados	archivo seguro

16.1. Builds de profiling

Una build de profiling se etiqueta explícitamente. No puede usarse para medir rendimiento final si Deep Profiling altera de forma material el resultado.

16.2. Logging

- no registrar secretos o datos personales;
- evitar spam por frame;
- incluir contexto e ID estable;
- diferenciar info, warning y error;
- no ocultar error con try/catch vacío;
- una build candidata no puede presentar errores recurrentes ignorados.

17. Nomenclatura de builds

17.1. Formato

CAC_v<appVersion>_<milestone>_<type>_Windows_x64_<YYYY-MM-DD>_Build<NNN>

Ejemplos:

```
CAC_v0.0.26_Sprint16_Dev_Windows_x64_2026-07-01_Build004
CAC_v0.3.0_H6_Candidate_Windows_x64_2026-07-XX_Build001
CAC_v0.4.0_Alpha_Private_Windows_x64_2026-XX-XX_Build002
CAC_v1.0.0_RC_Windows_x64_2027-XX-XX_Build003
```

17.2. Reglas

- ASCII, sin espacios;
- fecha de producción, no de publicación tardía;
- contador con tres dígitos;
- no usar “Final” salvo release aprobado; preferir Candidate o BuildNNN;
- no sobrescribir una carpeta ya validada;
- ZIP conserva el mismo basename;
- checksum usa <basename>.sha256;
- build record usa <basename>_Build_Record.md o ID equivalente.

17.3. Build ID interno

Formato recomendado:

S16-WIN-DEV-004
S17-WIN-QA-001
H6-WIN-CAND-001
v2.0-WIN-REL-001

El Build ID es corto; el nombre del artefacto es descriptivo. Ambos aparecen en el manifiesto.

18. Estructura de salida

```
Builds/  
└─ Windows/  
    └─ Development/  
        └─ <BuildName>/  
    └─ QA/  
        └─ <BuildName>/  
    └─ Candidates/  
        └─ <BuildName>/  
    └─ Demo/  
    └─ Release/  
    └─ Archive/
```

Builds/ permanece fuera de Git. Los registros, checksums, scripts y manifiestos sí se versionan en documentación o herramientas.

18.1. Separación de staging

Para candidatos:

```
Artifacts/  
└─ player/  
└─ logs/  
└─ tests/  
└─ evidence/  
└─ manifests/  
└─ checksums/
```

No empaquetar:

- saves personales;
 - Player.log de otra build;
 - capturas no relacionadas;
 - credenciales;
 - archivos de Unity Editor;
 - Library;
 - source code salvo paquete acordado;
 - builds anteriores dentro del ZIP.
-

19. Metadata y manifiesto de build

Cada build retenida debe tener un manifiesto legible por humanos y, cuando se automatice, por máquinas.

19.1. Campos mínimos

```

build_id: S16-WIN-DEV-004
application_version: 0.0.26
build_iteration: 4
commit_sha: <40-char-sha>
build_date_utc: 2026-07-01T00:00:00Z
unity_version: 6000.3.18f1
build_profile: Windows_Development
platform: Windows
architecture: x86_64
scripting_backend: Mono
compression: LZ4
development_build: true
schema_version: 2
content_version: unversioned
scenes:
  - Bootstrap
  - MainMenu
  - StoreInitial
test_summary:
  editmode: 1215/1215
  playmode: 70/70
gate: Sprint16
status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"
  "buildId": "H6-WIN-CAND-001",
  "applicationVersion": "0.3.0",
  "buildIteration": 1,
  "commitSha": "0000000000000000000000000000000000000000000000000000000000000000",
  "builtAtUtc": "2026-07-01T00:00:00Z",
  "unityVersion": "6000.3.18f1",
  "profile": "Windows_H6_Candidate",
  "platform": "Windows",
  "architecture": "x86_64",
  "backend": "Mono",
  "schemaVersion": 2,
  "scenes": ["Bootstrap", "MainMenu", "StoreInitial"],
  "tests": {
    "editMode": {"passed": 0, "failed": 0},
    "playMode": {"passed": 0, "failed": 0}
  },
  "artifactSha256": ""
}

```

53. Plantilla de release record

Release Record – <Version / Channel>

Approval

- Gate:
- Decision:
- Approved by:
- Date:

Artifact

- Build ID:
- File:
- Size:
- Storage:

Source

- Tag:
- Commit:
- Baseline documentation:

Compatibility

- Save schema:
- Supported previous versions:
- Upgrade path:
- Downgrade policy:

Distribution

- Channel/branch:
- Steam BuildID, if any:
- Publication time:

Known issues

Rollback

54. Historial de builds verificado

Hito	Versión	Tipo	Resultado	Evidencia retenida
Sprint 3	0.0.4	Development	PASS	record; sin ZIP/hash
Sprint 4	0.0.5	Development	PASS	record; sin ZIP/hash
Sprint 5	0.0.6	Development	PASS	record; sin ZIP/hash
Sprint 7	0.0.8	Development	PASS	warning Unity Services no
Sprint 8	0.0.9	Development	PASS	record operativo
Sprint 9	0.0.10	Development	PASS	ruta/tamaño/hash no regis
Sprint 10	0.0.11	Development	PASS	ruta/tamaño/hash/log no
Sprint 11	0.0.12	Development	PASS	record operativo
Sprint 12	0.0.13	Development	PASS	record operativo
Sprint 13	0.0.14	Development	PASS	record operativo
Sprint 14	0.0.15	Development	PASS	save/recovery validados
Sprint 15	0.0.16	Development	PASS	UI, slots y autosave valid
Sprint 16 preintegración	0.0.26	Development	PASS histórico	sustituido por la build de
Sprint 16 cierre	0.0.26	Development Windows x64	PASS	Golden Path, persistencia

54.1. Lecciones

- un build puede ser válido sin retener ZIP en cada sprint;
- los campos ausentes no deben inventarse;
- los hashes tienen más valor en hitos;
- PlayMode assemblies deben usar UNITY_INCLUDE_TESTS para no entrar al Player;
- los warnings de Unity Services son no bloqueantes mientras no exista dependencia;
- la transición a StoreInitial quedó respaldada por nueva evidencia en 0.0.26; el PASS previo no se reutilizó como sustituto.

55. Deuda y evolución recomendada

Prioridad inmediata

1. completar StoreInitial;
2. actualizar perfil después de conectar runtime;
3. generar build post-integración;
4. revisar Player.log;

5. crear registro completo;
6. abrir Sprint 17 solo tras cerrar Sprint 16.

Antes de H6

- perfil H6 Candidate;
- manifest de build;
- script mínimo de validación de escenas/versión;
- campaña de siete días;
- hardware y rendimiento;
- known issues congelados.

Antes de demo/Alpha

- comando de build reproducible;
- CI o ejecución scripted;
- content manifest;
- política de símbolos;
- crash reporting evaluado;
- localización y licencias;
- perfil sin Development;
- pruebas de actualización.

Antes de Release

- backend final;
 - firma;
 - SteamPipe;
 - branches/depots;
 - rollback;
 - soporte/hotfix;
 - seguridad y secretos;
 - archivo permanente.
-

56. Taxonomía de fallos de build

Los fallos deben clasificarse antes de corregirse. Una clasificación correcta evita aplicar soluciones destructivas, como borrar Library, cambiar paquetes o desactivar tests sin conocer la causa.

56.1. Fallo de compilación C#

Síntomas:

- CSxxxx en consola;
- assembly que no compila;
- tipo o namespace ausente;
- referencia circular;
- código de test incluido en Player.

Respuesta:

1. conservar el primer error y su contexto;
2. identificar assembly y cambio que lo introdujo;
3. revisar .asmdef, define constraints y referencias;
4. corregir el error raíz antes de los secundarios;
5. ejecutar compilación y tests dirigidos;

6. no cambiar paquetes para ocultar un error de código.

El incidente histórico de S0.9 mostró que un assembly PlayMode sin `UNITY_INCLUDE_TESTS` puede entrar en el Player y hacer fallar la build. La corrección correcta fue ajustar el assembly, no eliminar las pruebas.

56.2. Fallo de importación o asset

Síntomas:

- GUID faltante;
- material rosa;
- referencia Missing;
- FBX o textura que no importa;
- prefab inválido;
- catalog validation fallida.

Respuesta:

- comprobar .meta;
- revisar origen del asset;
- validar que no se movió fuera de Unity;
- reimportar solo el asset o carpeta afectada;
- comparar contra el commit;
- no regenerar GUID manualmente;
- verificar licencias y tamaño.

56.3. Fallo de Scene List

Síntomas:

- build arranca en escena equivocada;
- Scene couldn't be loaded;
- MainMenu no encuentra StoreInitial;
- escena presente pero deshabilitada;
- TestLab aparece en distribución.

Respuesta:

1. inspeccionar Build Profile y lista global;
2. comparar ruta y GUID;
3. confirmar Bootstrap en índice 0;
4. revisar la configuración runtime que contiene el nombre de escena;
5. ejecutar smoke externo;
6. actualizar QA y ADR si cambió el contrato.

56.4. Fallo de Player distinto del Editor

Posibles causas:

- stripping;
- orden de inicialización;
- paths;
- case sensitivity de nombres;
- código protegido por `UNITY_EDITOR`;
- assets no incluidos;
- diferencias de timing;
- serialización;
- plugins;

- permisos de archivo.

Respuesta:

- conservar `Player.log`;
- reproducir con el mismo artefacto;
- activar una build diagnóstica separada si hace falta;
- no modificar el candidato original;
- escribir test que capture el defecto cuando sea viable;
- comprobar que la corrección funciona en Player, no solo en Editor.

56.5. Fallo de empaquetado o hash

Síntomas:

- ZIP incompleto;
- doble carpeta raíz;
- hash no coincide;
- ejecutable bloqueado durante compresión;
- archivo de otra build incluido.

Respuesta:

- cerrar procesos;
- borrar solo el paquete fallido, no la carpeta fuente validada;
- volver a empaquetar desde staging limpio;
- probar extracción;
- recalcular hash;
- incrementar Build iteration si el artefacto ya se comunicó.

56.6. Fallo de almacenamiento

- disco lleno;
- ruta demasiado larga;
- permisos insuficientes;
- antivirus bloquea archivo;
- backup no disponible.

El preflight debe medir espacio y verificar escritura. No se debe aprobar un artefacto cuyo único original esté en una ruta temporal.

57. Entorno y hardware del build

57.1. Registro del entorno

Para builds de hito conservar:

Campo	Ejemplo
OS	Windows, edición y build
CPU	modelo
RAM	capacidad
GPU/driver	si afecta shader/perfil
Disco	tipo y espacio disponible
Unity	versión/revisión
Hub	opcional

Campo	Ejemplo
Git	versión
Locale	idioma/región
Zona horaria	UTC/local
Antivirus	observación si interviene

No se pretende que cada build ordinaria tenga un inventario exhaustivo. El objetivo es poder explicar diferencias de rendimiento, paths, compresión o importación.

57.2. Hora y fecha

- manifest usa UTC;
- nombres pueden usar fecha local acordada;
- registros indican zona horaria;
- no confiar solo en timestamp del sistema de archivos;
- sincronizar reloj antes de firma o publicación.

57.3. Máquina de build

Antes de release conviene una máquina o runner controlado:

- software mínimo;
- acceso restringido;
- toolchain fijado;
- sin plugins personales;
- almacenamiento cifrado de secretos;
- logs preservados;
- capacidad de reconstrucción.

Mientras se use la máquina de desarrollo, registrar cualquier diferencia material y evitar builds con herramientas experimentales activas.

58. Procedimiento de bump de versión

58.1. Bump ordinario de sprint

1. revisar el charter;
2. decidir versión objetivo;
3. comprobar última versión publicada;
4. actualizar `PlayerSettings.bundleVersion`;
5. actualizar registro de sprint;
6. ejecutar test que lea la versión, si existe;
7. verificarla en build externa;
8. no cambiarla durante la aceptación salvo que se invalide la build.

58.2. Bump por hotfix

- partir de la versión afectada;
- incrementar PATCH;
- conservar compatibilidad de schema o documentar migración;
- actualizar release notes;
- no incorporar features de la rama principal sin necesidad;
- probar actualización desde el artefacto anterior.

58.3. Bump de schema

La secuencia correcta es:

```
definir nueva forma
→ implementar migrador
→ conservar reader anterior
→ crear fixtures
→ tests de migración
→ incrementar constante de schema
→ build
→ validación de saves
```

Incrementar primero la constante y “arreglar después” crea una ventana en la que el programa puede escribir datos sin lector fiable.

58.4. Bump documental

- actualizar front matter;
- conservar fecha;
- añadir changelog;
- generar hash;
- no cambiar la versión de aplicación si no cambió el Player.

58.5. Bump de contenido

Cuando exista contentManifestVersion:

- calcularlo a partir de catálogo aprobado o incrementarlo mediante procedimiento;
- vincularlo al manifest de build;
- probar IDs faltantes;
- documentar cambios de balance significativos;
- no sustituir schema por content version cuando cambia estado persistente.

59. Matriz de artefactos por gate

Artefacto	Sprint ordinario	Sprint 16	Sprint 17	H6	Demo/Release
Build record	sí	sí	sí	sí	sí
SHA de commit	sí	sí	sí	sí	sí
Test summary	sí	completo	completo	completo	completo
Player.log	condicional	obligatorio	obligatorio	obligatorio	obligatorio
Golden Path	según alcance	obligatorio	obligatorio	obligatorio	obligatorio
ZIP	opcional	recomendado cierre	sí	obligatorio	obligatorio
Manifest	objetivo	recomendado	sí	obligatorio	obligatorio
Known issues snapshot	básico	sí	sí	sí	sí
Hardware/performance	no siempre	smoke	obligatorio	obligatorio	obligatorio
Capturas/vídeo	opcional	aprobación visual	recomendado	obligatorio	marketing
Tag	no	no por defecto	decisión de hito	si se publica	sí
Release record	no	no por defecto	interno	sí	sí
Third-party notices	no	revisión	revisión	revisión	obligatorio
Rollback plan	no	fallback escena	sí	sí	obligatorio

59.1. Regla de proporcionalidad

La evidencia aumenta con el coste de distribución y recuperación. No se exige ceremonia de release a cada build local, pero tampoco se permite usar una build local como candidato sin completar sus artefactos.

60. Escenarios de migración y actualización

60.1. Nueva instalación

Validar:

- arranque sin carpeta previa;
- creación de directorio de datos;
- MainMenu y slot vacío;
- opciones por defecto;
- primer save;
- salida limpia.

60.2. Actualización compatible

- instalar o sustituir Player conservando datos;
- iniciar;
- detectar saves existentes;
- cargar cada slot soportado;
- guardar con schema vigente;
- reiniciar;
- comprobar equivalencia.

60.3. Actualización con migración

- backup previo;
- cargar fixture antiguo;
- migrar en memoria;
- validar;
- escribir nuevo primario;
- preservar backup anterior según política;
- registrar migración;
- impedir doble migración.

60.4. Save futuro

Una build antigua que recibe un schema futuro debe:

- identificar incompatibilidad;
- no sobrescribir;
- no “reparar” destructivamente;
- mantener backup;
- mostrar mensaje útil;
- permitir volver al menú/salir.

60.5. Cambio de Application Identifier

Si alguna vez es imprescindible:

- mapear ruta antigua y nueva;

- copiar, no mover inicialmente;
- validar checksums;
- manejar conflictos;
- ofrecer rollback;
- probar perfiles de usuario sin permisos elevados;
- documentar la transición.

60.6. Demo a juego completo

- resolver App/depots;
 - decidir si comparte carpeta de save;
 - marcar origen de la partida;
 - migrar contenido limitado;
 - evitar referencias a assets ausentes;
 - validar logros/Steam Cloud si existen;
 - comunicar qué progreso se conserva.
-

61. Revisión formal de un candidato

61.1. Reunión o acto de gate

Aunque el desarrollo sea individual, realizar una revisión separada:

1. leer objetivo;
2. identificar artefacto sin reconstruirlo;
3. confirmar SHA/hash;
4. revisar pruebas;
5. ejecutar o revisar Golden Path;
6. inspeccionar `Player.log`;
7. revisar defectos;
8. revisar saves y rendimiento;
9. decidir PASS, deuda o FAIL;
10. registrar siguiente acción.

61.2. Preguntas de rechazo

- ¿el artefacto puede no corresponder al SHA?;
- ¿hay un S0/S1?;
- ¿se perdió evidencia?;
- ¿el profile contiene escenas incorrectas?;
- ¿el save puede dañarse?;
- ¿el candidato fue modificado después del hash?;
- ¿la aprobación visual sigue pendiente?;
- ¿se utilizó una build de profiling como final?;
- ¿las licencias están sin resolver?;

Un “sí” material produce FAIL o bloqueo, no una observación decorativa.

61.3. Cierre

Después del PASS:

- bloquear el artefacto;
- copiar a archivo;
- publicar tag/release cuando proceda;
- actualizar baseline;

- marcar builds anteriores como superseded, no borrarlas si son hitos;
 - comunicar canal y known issues.
-

62. Runbook de recuperación

62.1. La build candidata no arranca

- no reconstruir sobre la misma carpeta;
- conservar logs;
- probar en la máquina de build y otra máquina si existe;
- verificar dependencias y cuarentena antivirus;
- volver al último candidato aprobado;
- abrir defecto S1;
- producir nueva iteración después del fix.

62.2. El hash publicado no coincide

- detener distribución;
- comparar archivo y fuente;
- retirar checksum incorrecto;
- determinar si cambió el ZIP o el registro;
- no “corregir” el hash sin explicar el incidente;
- publicar artefacto/registro nuevo y dejar trazabilidad.

62.3. Se incluyó un secreto

- retirar artefacto;
- revocar/rotar secreto;
- limpiar historial cuando proceda;
- comprobar descargas y logs;
- abrir incidente de seguridad;
- producir build nueva;
- no confiar en borrar solo el archivo visible.

62.4. Corrupción de saves tras update

- retirar/promover rollback según seguridad;
- preservar saves afectados;
- no pedir al usuario que los sobrescriba;
- identificar schema/build;
- construir herramienta o hotfix de recuperación;
- probar con copias;
- comunicar workaround seguro.

62.5. Build profile dañado

- comparar con Git;
- restaurar asset y .meta juntos;
- no recrear a ciegas si cambia GUID;
- validar escenas;
- ejecutar smoke;
- registrar por qué se dañó.

62.6. Paquete o Unity cambió accidentalmente

- detener build;
- revisar manifest, lock y ProjectVersion;
- restaurar baseline;
- reimportar;
- ejecutar suite;
- no aceptar el cambio como “actualización automática”.

Anexo A. Rutas relevantes

ProjectSettings/ProjectVersion.txt
ProjectSettings/ProjectSettings.asset
ProjectSettings/EditorBuildSettings.asset
Assets/_Project/Settings/BuildProfiles/Windows_Development.asset
Packages/manifest.json
Packages/packages-lock.json
Documentation/10_Development_Records/Builds/
Documentation/10_Development_Records/Sprint_*/QA/*Build*Record.md
Documentacion/08_QA_Testing_Matrix.xlsx

Anexo B. Valores protegidos

Cambiar cualquiera requiere revisión formal y, en varios casos, ADR:

- Unity 6000.3.18f1;
- Windows x64 como plataforma inicial;
- Bootstrap como entrada;
- ApplicationRoot único;
- Application Identifier;
- company/product name;
- backend;
- schema;
- ruta/ownership de saves;
- orden de escenas;
- política de tags;
- perfil Release;
- Steam IDs futuros;
- firma y certificados.

Anexo C. Definiciones

Término	Definición
Artefacto	archivo o conjunto producido por build
Baseline	conjunto aprobado de código, configuración, pruebas y documentos
Build	proceso y/o resultado compilado, según contexto
Build ID	identificador único de una iteración
Candidate	artefacto sometido a gate
Channel	destino de distribución
Checksum	hash de integridad
Clean build	build desde estado limpio/controlado
Depot	contenedor de archivos en Steam
Manifest	metadata de identidad y configuración

Término	Definición
RC	Release Candidate
Schema	versión de estructura persistente
SHA	identificador de commit o hash de archivo, según campo
Smoke	recorrido mínimo de viabilidad
Superseded	reemplazado por artefacto posterior

Anexo D. Ruta final

Documentacion/
└─ 11_Build_y_Versioning_Guide.md

Estado del documento: fuente vigente para builds y versionado. Debe actualizarse al cerrar Sprint 16, congelar la versión de Sprint 17, producir el candidato H6 o adoptar automatización, IL2CPP, firma o Steam.

Actualización de baseline y build W0

Estado operativo vigente tras W0

Elemento	Estado vigente
Baseline de entrada	main@efbc1a885a1d6819f764ec8cff8a465ad1986661 / aplicación 0.0.26
W0 documental	COMPLETED / DOCUMENTAL PASS
Decisiones funcionales abiertas	0
Sprint17_Phase1	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
H6	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
Vertical Slice	ACCEPTED / IMPLEMENTED / VALIDATED / CLOSED

Baseline de remediación

Campo	Valor
Rama	main
Commit	efbc1a885a1d6819f764ec8cff8a465ad1986661
Versión de aplicación	0.0.26
Alcance	entrada documental para W1-W8
Build H6	no creada / no aprobada

Cada ola debe registrar commit, versión, pruebas afectadas y compatibilidad de save. W8 exige build Windows x64 externa, Golden Path, campaña de siete días, cierre/reapertura, Player.log y reconciliación de inventario/economía. Ningún artefacto de Sprint 16 se renombra como build H6.

Regla de cierre y no propagación

- W0-W8 están completadas, validadas y cerradas en la baseline 0.0.26.
- Sprint17_Phase1 y W8 están completadas, validadas y cerradas.
- H6 y la Vertical Slice disponen de aceptación propia y están cerrados sobre 0.0.26.
- La Vertical Slice queda cerrada; los sistemas Post-H6 se gestionarán como evolución posterior mediante control de cambios.

-